

SIH26003

Cognitive Care, Monitoring and Welfare Platform

Industry-Grade Product and Architecture Specification

Prepared from the supplied 56-page architecture conversation
with final-state engineering decisions consolidated into one buildable blueprint.

Version 1.0 | 07 September 2026

Architecture in 150 Words or Less

The platform is offline-first and clinic-provisioned, with three coordinated surfaces. The Android patient client is Kotlin + Jetpack Compose and runs cognitive games, medication reminders, on-device speech, and a local RL-IRT adaptive engine. Room Database stores durable app-data telemetry in FIFO order; WorkManager synchronizes only when connectivity exists. Audio is captured with AudioRecord, resampled from native 44.1 kHz mono to 16 kHz mono, then fed to on-device Bhashini IndicConformer while the same 16 kHz stream is encoded with native MediaCodec as AMR-WB at 15.85 kbps. Firebase Auth, Firestore, Storage and Cloud Functions provide the cloud control plane; Custom Claims enable doctor access. The React + Vite dashboard reads Firestore and securely retrieves protected audio with the Firebase JS SDK, GCS CORS configuration and browser-side WASM decoding. Exact medication/game reminders use AlarmManager; network work uses constrained WorkManager. The system is designed for weak 2G/3G, budget hardware, delayed synchronization, and welfare escalation.

Source basis: final-state architecture is documented in the supplied conversation at pp. 48-56, with earlier correction rounds consolidated in the decision history in this document.

1. Executive Summary

The SIH26003 platform is designed as a clinically supervised cognitive-care companion for people living with dementia or cognitive impairment, operating under severe connectivity, hardware, literacy, and deployment constraints. The first version is not a self-onboarding consumer app: the supplied architecture mandates clinic or hospital setup after a patient has reached a doctor and caregiver. A future caregiver-only edition is anticipated for educated users, but it is outside the current deployment contract.

The product combines routine cognitive activities, medication adherence support, passive/active telemetry, adaptive psychometric selection, offline persistence, speech accessibility, and a doctor-facing web dashboard. Its defining property is graceful degradation: the patient experience continues offline, data waits safely on-device, and cloud synchronization resumes when a connected network is available.

The uploaded source explicitly locks the architecture around Kotlin/Jetpack Compose, React/Vite, Firebase, Room, WorkManager, AlarmManager, an on-device RL-IRT hybrid, Bhashini IndicConformer and FastPitch/HiFi-GAN, dynamic language packs, AMR-WB voice storage, 16 kHz resampling, Custom Claims, schema versioning, and a 50 MB storage guard.

1.1 Primary Uses

- Provide a simple, repetitive and accessible daily routine for cognitive exercises and medication reminders.
- Collect structured game performance and timing data suitable for longitudinal review by an authorized clinician.
- Provide one speech-response question per game, rather than making every item an audio upload, to preserve bandwidth and allow a longer response window.
- Operate without continuous internet access, storing telemetry as durable application data until synchronization is possible.
- Provide a welfare signal when expected activity does not occur, with server-side escalation to caregivers/doctors after configured inactivity windows.
- Allow clinicians to review trends, adherence, alerts, sessions and selected voice evidence from a React web dashboard.

1.2 Non-Goals

- The app is not a substitute for diagnosis, prescribing, emergency response, or clinician judgment.
- The platform does not use a heavy generative LLM on the device.
- The mobile app does not depend on a permanent cloud connection for core patient operation.
- The current version does not expose unrestricted patient self-enrollment; clinic/hospital provisioning is a hard requirement.

2. Product Definition and User Model

2.1 Roles and Trust Boundaries

Role	Primary responsibility	Allowed product surface	Trust boundary
Patient	Complete games, respond to reminders, take medication, provide limited speech response	Android app	No access to other patients or clinician administration
Caregiver	Provision patient routine, observe adherence, respond to welfare prompts	Android caregiver/admin flows; future caregiver-only setup	Can manage assigned patient, not clinician-wide data
Doctor	Review longitudinal telemetry, scores, alerts, sessions and audio evidence	React/Vite web dashboard	Authenticated clinician with role claim plus patient assignment policy
Emergency contact	Receive welfare escalation	Notification/SMS path	No dashboard data access unless explicitly authorized
System services	Sync, heartbeat, schema adaptation, storage management	Firebase Cloud Functions / background workers	Service credentials only; no direct patient UI access

2.2 Provisioning Contract

1. Patient arrives at a clinic/hospital and is linked to a doctor and caregiver.
2. Clinic device setup selects the language pack and downloads the required mobile speech models over high-speed clinic Wi-Fi.
3. Clinician/caregiver completes identity, contact, medication schedule, reminder schedule and initial game configuration.
4. Android app verifies that required models are present, that local storage is above the safety threshold, and that the device is registered.
5. Patient leaves the clinic with an offline-capable configuration and an assigned care relationship.

The supplied source explicitly states clinic/hospital provisioning, dynamic language packs, and a future no-doctor caregiver-only edition.

2.3 Patient UX Principles

- One primary action per screen; avoid dense menus.
- High contrast, large touch targets, forgiving timing, clear progress indication.
- Every text prompt can be spoken using local TTS.
- For illiterate users, each option is spoken when selected, the selected option is highlighted yellow/green, and a single large green Submit button commits the response.
- Never use color alone to communicate correctness, status or emergency state; pair color with shape, text-to-speech or iconography.
- No patient-facing technical errors such as token, network, database or storage exceptions.

3. Android Mobile Application Architecture

The reference mobile implementation is a Kotlin Android application using Jetpack Compose for the UI. The architecture should be layered so that UI, domain logic, local persistence, speech, adaptive selection, reminders and synchronization are independently testable.

3.1 Recommended Module Boundaries

Layer/module	Responsibility
app	Application startup, dependency graph, navigation shell, lifecycle handling
ui	Compose screens, accessibility, state rendering, voice/readout controls
domain	Game rules, scoring rules, medication state transitions, session orchestration
adaptive	RL/contextual-bandit orchestration plus IRT item selection
speech	TTS/STT facades, language pack validation, TFLite inference lifecycle
audio	AudioRecord capture, resampling, AMR-WB MediaCodec encoding, file lifecycle
data-room	Room entities, DAOs, migrations, durable telemetry queue
sync	CoroutineWorker state machine, upload checkpoints, connectivity constraints
reminders	AlarmManager schedules and high-priority BroadcastReceiver handlers
security	Firebase Auth session, device registration and local authorization state
diagnostics	structured local logs, queue metrics, model health, storage health

3.2 Screen Map

- Provisioning/Setup: clinic activation, patient linkage, language pack state, device readiness.
- Home: today card, next reminder, start recommended game, medication action, caregiver access.
- Game launcher: only games currently allowed by adaptive scheduler and daily plan.
- Game screen: question, optional spoken prompt, selectable answer area, large green Submit button.
- Speech response screen: one designated speech task, 15-30 second window, record/stop status, retry locally if capture fails.
- Medication screen: current medication, due state, Taken / Snoozed / Unable-to-take workflow.
- Caregiver status screen: connectivity, last sync, storage health, model pack health, pending queue count.
- Clinician pairing/admin screen: protected, not part of normal patient navigation.

3.3 Storage Safety Gate

Before a game starts, the app must check available physical storage. The locked design uses a 50 MB free-space threshold. Below that threshold, the game is blocked and a high-contrast, TTS-enabled warning is shown to the caregiver; the app can launch Android storage management so the caregiver can remove unrelated files. This is specifically intended to prevent mid-session Room/SQLite failure.

4. Cognitive Game System

The supplied conversation did not provide a canonical named catalog of games. Therefore, this section deliberately separates two things: (A) the architecture constraints that are source-derived and locked, and (B) a proposed corporate-grade game catalog created for this specification so that an implementation team has concrete content to build. The proposed games are product ideas, not claims that the source conversation already implemented them.

4.1 Locked Game Architecture

- Games are predominantly MCQ-based to reduce telemetry and audio payload volume.
- Exactly one targeted speech-response question is permitted per game/session; the window is 15-30 seconds.
- Each question produces structured item-level telemetry: item id, domain, difficulty, response, correctness, latency, hint/retry state and session context.
- IRT selects item difficulty/content; the reinforcement-learning or contextual-bandit layer selects the next game/item strategy to balance engagement and information value.
- All game state must be recoverable locally from Room so the app can resume after interruption without creating duplicate telemetry.

4.2 Proposed Game Catalog

Game	Primary domain	Gameplay	Input	Adaptive / audio note
1. Picture Recall	Memory / delayed recall	Show 3-6 everyday pictures, hide them, ask which picture was present. Adaptive difficulty: item count, distractor similarity, delay.	MCQ	Optional speech question: name one remembered object.
2. Object Matching	Associative memory	Match household object to its typical use or pair identical objects.	MCQ	TTS reads object/use choices.
3. Sequence Memory	Working memory	Display a short visual sequence, then ask the patient to choose the sequence in the correct order.	MCQ	Increase sequence length slowly.
4. Spatial Grid Recall	Visuospatial memory	Highlight positions on a grid, clear them, ask which cells were highlighted.	MCQ	Adaptive grid size and target count.
5. Odd One Out	Attention / categorization	Four objects or symbols are shown; choose the one that differs by the target rule.	MCQ	Vary rule complexity.
6. Category Sorting	Executive function	Choose which object belongs to a named category from visual choices.	MCQ	Use culturally familiar local objects.
7. Stroop-like Color Choice	Selective attention	Spoken or visual color word paired with a mismatching ink color; select target	MCQ	Use simplified, accessibility-safe version.

		attribute.		
8. Trail-Path Choice	Executive / sequencing	Choose the next number/letter target in an ordered visual path without requiring handwriting.	MCQ	Keep motor load low.
9. Clock Orientation	Orientation / visuospatial	Select the clock showing a requested time, or identify morning/evening.	MCQ	Use large analog clocks with clear numerals.
10. Day and Date Orientation	Orientation	Choose correct day, date, month, or time-of-day concept.	MCQ	Record confidence and response latency.
11. Face-Name Pairing	Associative memory	Pair a simple illustrated/persona face with a first name, then recall the pair later.	MCQ	Use consent-safe synthetic personas, not real staff.
12. Route Memory	Spatial memory	Show a short route through 3-5 landmarks, then ask which route was correct.	MCQ	Use local visual landmarks where suitable.
13. Number Span	Working memory	Hear or see a short digit sequence; select the exact sequence from options.	MCQ	Forward span first; reverse span only for appropriate stages.
14. Calculation Choice	Processing / executive function	Simple everyday arithmetic such as change, counting objects, or time differences.	MCQ	Adaptive difficulty and culturally realistic currency examples.
15. Change Detection	Visual attention	Show a scene twice with one small change; ask which element changed.	MCQ	Increase delay and subtlety.
16. Pattern Completion	Abstract reasoning	Choose the missing element from a simple visual pattern.	MCQ	Use large, uncluttered patterns.
17. Everyday Decision Game	Executive function	Choose the correct action in a routine scenario, such as what to do before taking medicine.	MCQ	Score decision accuracy, not compliance alone.
18. Delayed Word Recognition	Language / memory	Introduce a small spoken word set, later ask which word was in the original set.	MCQ + one speech trial	TTS supports non-readers.
19. Sound-Source Identification	Auditory attention	Play a short familiar sound and choose its source.	MCQ	Store audio assets locally in language pack.
20. One-Minute Story Recall	Language / episodic memory	Play/read a short local-language story, then ask 3-5 comprehension/recall questions. One selected question may use speech response.	MCQ + one speech trial	Keep story length controlled.

21. Emotion Recognition	Social cognition	Select the facial expression matching a simple emotion word or scenario.	MCQ	Use culturally neutral illustrations.
22. Inhibition Tap	Response inhibition	In a stream of shapes, tap only when the target appears.	Tap	Measure commission errors and reaction time.
23. Reaction Choice	Processing speed	Choose the matching symbol as quickly as practical.	Tap	Do not over-weight speed for users with motor impairment.
24. Paired Associates	Learning / recall	Learn object-location or object-word pairs, then recall them later.	MCQ	Adaptive pair count and delay.

4.3 Game Session Contract

A session should be a deterministic container with a session id created locally before the first item. The session contains the selected game version, schema_version, patient/device reference, planned item list or selection seed, per-item responses, timing, score summary and, when applicable, one speech artifact. The app writes the completed session to Room before scheduling upload so a power loss after the game never destroys the record.

4.4 Telemetry Fields for Games

Field group	Required data
Identity	patient_uid, device_id, session_id, game_id, game_version
Schema	schema_version, client_version, model_pack_version
Timing	session_started_at, item_started_at, item_completed_at, duration_ms
Response	selected_option_id, correct_option_id, is_correct, attempts, hint_used
Psychometric	domain, item_difficulty, latent_estimate_before, latent_estimate_after, information_value
Adaptive	policy_id, context_vector_id or compact context fields, action_selected, exploration_flag
Speech	has_audio, audio_storage_path, recording_duration_ms, transcript_if_stored
Reliability	local_sequence_no, upload_state, created_at_device, sync_attempts

The source explicitly describes Firestore as the structured store for psychometric telemetry and session logs, while Firebase Storage is reserved for binary audio and model packs.

5. On-Device Adaptive Intelligence: RL-IRT Hybrid

The source architecture uses a local RL-IRT hybrid: reinforcement learning/contextual bandit logic manages engagement or selection policy, while Item Response Theory performs psychometric item selection. The entire engine is intended to run in Kotlin without a generative language model.

5.1 Responsibilities

Component	Input	Output	Constraints
IRT item selector	Patient latent estimate, item bank parameters, recent responses	Next item candidate/ranking	Deterministic, low compute, local only
Contextual bandit / RL layer	Engagement context, recent completion/skip/error signals	Game/item action or policy choice	Small state/action space; bounded exploration
Scoring service	Item response + item parameters	Domain score / latent estimate update	Clinician-facing interpretation required
Policy guardrails	Safety, fatigue, recent repetition, accessibility profile	Allowed action set	Hard rules override adaptive policy
Telemetry compressor	Full local event stream	Compact Firestore-ready session metadata	No clinical fields silently dropped

5.2 Suggested Domain Model

- Memory: immediate recall, delayed recall, associative learning.
- Attention: selective attention, sustained attention, visual change detection.
- Executive function: inhibition, sequencing, categorization, decision selection.
- Visuospatial: spatial recall, clock orientation, route memory.
- Language: recognition/comprehension tasks plus the single speech task when selected.
- Processing: reaction time and choice speed, interpreted cautiously for motor limitations.
- Orientation: day/date/time and routine-context tasks.

5.3 Adaptive Selection Order

6. Apply hard constraints: patient fatigue, accessibility, daily plan, recently administered items, and safety exclusions.
7. Estimate current domain/item state from IRT and recent history.
8. Generate a small candidate set of eligible items.
9. Use the contextual bandit/RL policy to rank or choose among candidates under a bounded exploration policy.
10. Execute the item and record response/timing.
11. Update local psychometric state immediately so the next offline interaction remains adaptive.
12. Persist the session state to Room before any network work is attempted.

6. Speech, TTS and Audio Engineering

The final architecture keeps speech local. Bhashini IndicConformer is used for STT; FastPitch + HiFi-GAN is used for TTS; the runtime is TFLite with select TensorFlow operations. Language packs are self-hosted in Firebase Storage and are downloaded at clinic provisioning time. The source specifically moved away from relying on a nonexistent external mobile-binary CDN and added tensorflow-lite-select-tf-ops for complex speech graphs.

6.1 Single Speech Question Rule

A game contains many MCQ interactions but only one targeted speech-response question. The recording window is 15-30 seconds, deliberately enlarged to accommodate slower cognitive processing. This was a direct correction from the earlier 5-second limit. The change reduces data volume while protecting speech capture quality.

6.2 Final Capture Pipeline

13. Initialize AudioRecord at a native device rate, with the final source architecture specifying 44.1 kHz mono as the baseline capture target to avoid Audio HAL initialization failures.
14. Read 16-bit PCM into a bounded in-memory buffer.
15. Run pure Kotlin linear interpolation resampling from 44.1 kHz to exactly 16 kHz.
16. Feed the 16 kHz mono stream to the Bhashini TFLite STT path.
17. Feed the same 16 kHz mono stream to MediaCodec configured for AMR-WB at 15.85 kbps.
18. Write AMR-WB frames directly to the application file stream with the required `#!AMR-WB` newline header; do not use MediaMuxer for raw AMR-WB output.
19. Close and atomically attach the completed audio file to the Room telemetry record before the sync worker sees it.

The final source architecture explicitly states 44.1 kHz capture, Kotlin linear interpolation to 16 kHz mono, MediaCodec AMR-WB at 15.85 kbps, direct FileOutputStream framing, and a roughly 60 KB 30-second voice file.

6.3 Resampling Math

For an input sample rate $F_{in} = 44,100$ Hz and output rate $F_{out} = 16,000$ Hz, the interpolation step is $F_{out} / F_{in} = 0.3628118$ in output-per-input terms, or an input-position increment of $F_{in} / F_{out} = 2.75625$ for each output sample. For output sample i , compute $pos = i * 2.75625$, $alpha = pos - \text{floor}(pos)$, then linearly interpolate between the adjacent input samples. The implementation must generate a new 16 kHz mono ShortArray rather than relabeling the 44.1 kHz buffer.

6.4 Why AMR-WB Is Final

Earlier iterations attempted 128 kbps AAC with a target of roughly 15 KB. The source audit calculated that 15 seconds at 128 kbps would be about 240 KB and 30 seconds about 480 KB, contradicting the original 15 KB target. The design therefore moved to AMR-WB at 15.85 kbps, giving about 60 KB for 30 seconds while maintaining speech intelligibility.

7. Dynamic Language Packs and On-Device ML

Language support is delivered as a versioned pack installed at the clinic. The pack contains the required TTS/STT mobile binaries, metadata, checksums and language configuration. Pack size is expected to be roughly 150-300 MB in the supplied architecture, so field devices receive the pack before deployment over high-speed clinic Wi-Fi.

7.1 Build and Distribution Pipeline

20. Download the required Bhashini/AI4Bharat source checkpoints on a controlled developer workstation.
21. Convert/export models to the target mobile graph format.
22. Quantize to INT8 or FP16 according to the validated accuracy/size trade-off.
23. Validate required operators against the Android runtime, including select TensorFlow operations.
24. Run device-level speech regression tests for the target languages and acoustic conditions.
25. Package each language pack with explicit version, model hash, input/output tensor specification and minimum app version.
26. Upload the pack to a protected Firebase Storage path reserved for model assets.
27. At clinic provisioning, authenticate the device, download the pack, verify checksum, unpack it into durable application storage and mark the pack as ready in Room.

7.2 Model Compatibility Gate

- No model pack is considered installed until checksum, model signature/version and operator validation succeed.
- App startup checks that the installed pack matches the expected language and app-compatible version.
- Failed or partial downloads are discarded rather than treated as valid.
- Old packs remain available until the replacement pack passes validation, enabling safe rollback.

8. Offline Persistence, FIFO Queue and Synchronization

The local queue is durable application data, not disposable cache. The source explicitly states that RoomDB is stored in internal app-data so ordinary cache clearing does not erase telemetry. The queue is FIFO and is drained by WorkManager when connectivity exists.

8.1 Room Entities

Entity	Purpose	Key fields
PatientLocal	Local patient configuration	patientUid, displayProfileRef, language, timezone, schemaVersion
GameSession	Completed game summary	sessionId, patientUid, gameId, startedAt, completedAt, score, schemaVersion
TelemetryPackage	Upload unit / FIFO record	queueId, sequenceNo, payloadJson, audioLocalPath, uploadState, storagePath, retryCount
MedicationEvent	Taken/missed/snoozed event	eventId, medicationId, scheduledAt, actionAt, action, schemaVersion
ReminderSchedule	Local reminder definition	scheduleId, type, localTime, enabled, nextTriggerAt
ModelPack	Installed language model state	packId, language, version, checksum, localPath, verified
DeviceState	Operational telemetry	batteryBand, storageFree, networkState, appVersion, lastLocalActivityAt

8.2 Upload State Machine

State	Meaning	Allowed next state
PENDING	Nothing uploaded	AUDIO_UPLOADED or COMPLETED for metadata-only payload
AUDIO_UPLOADED	Binary safely uploaded; URL/path checkpoint stored locally	COMPLETED
COMPLETED	Firestore metadata confirmed	Terminal
RETRY_WAIT	Transient failure recorded	PENDING or AUDIO_UPLOADED
FAILED_PERMANENT	Payload invalid or repeatedly rejected by policy	Manual diagnostic path

8.3 Idempotent Retry Contract

28. Worker reads the head-of-queue record only after connectivity is reported by WorkManager.
29. Worker ensures a current Auth token by awaiting getIdToken(false) before protected writes.
30. If state is PENDING and audio exists, upload audio to the deterministic Storage path for that session.
31. Immediately persist the returned storage reference locally as AUDIO_UPLOADED.
32. Write Firestore metadata containing the same session id, storage path and schema_version.
33. On network drop during metadata write, return Result.retry() without re-uploading the audio.
34. On restart, read the persisted upload_state and resume from the checkpoint.
35. Only after successful metadata acknowledgement is the queue record deleted or marked completed.

The source describes the same state-machine pattern: persist upload_state after Storage success, then skip the Storage upload on retry and resume at Firestore.

9. Background Execution and Battery Strategy

A major correction in the supplied architecture was to separate time-critical clinical reminders from network synchronization. AlarmManager handles exact medicine/game reminders; WorkManager handles network-dependent synchronization and is constrained to NetworkType.CONNECTED. This prevents the sync engine from repeatedly waking a device in a zero-signal region.

9.1 AlarmManager Responsibilities

- Schedule exact medicine reminders where exact timing is required.
- Schedule exact intended-activity prompts such as a planned cognitive test.
- Use a high-priority BroadcastReceiver to enter the reminder UX.
- Keep reminder execution independent from network availability.

9.2 WorkManager Responsibilities

- Use OneTimeWorkRequest for queue draining.
- Use NetworkType.CONNECTED as the required network constraint.
- Use exponential backoff starting around 15 minutes for transient failures.
- Use enqueueUniqueWork for the queue and for any scheduled inactivity signal so multiple missed windows do not create an unbounded task herd.
- Never poll cellular connectivity manually from a background loop.

9.3 Alive-in-the-Cage Welfare Signal

The conversation introduced a deliberate "alive but inactive" concept: when an intended activity is missed, a lightweight signal is scheduled 15-30 minutes later. The final architecture should implement this as a local missed-activity marker plus a connectivity-constrained WorkManager signal. The server-side heartbeat remains authoritative: it compares the last known activity/sync timestamp to the configured welfare policy and escalates if the patient appears offline for too long. The alert is intentionally allowed to be a false positive because its purpose is a physical welfare check, not diagnosis.

The source records the 15-30 minute inactivity ping and server-side heartbeat concept.

10. Firebase Architecture

Firebase is the cloud control plane. Firestore stores structured metadata and telemetry; Firebase Storage stores binary patient audio and model packs; Firebase Authentication supplies user identity; Cloud Functions implement server-side verification, custom claims, schema adaptation and welfare monitoring; Firebase Cloud Messaging/SMS are used for notification routing where configured.

10.1 Reference Firestore Schema

Collection	Document id	Purpose	Representative fields
users	uid	Identity and role	role, displayName, disabled, createdAt, lastSeenAt
patients	patientUid	Patient master record	caregiverUid, clinicId, primaryDoctorUid, language, timezone, deviceId
careTeams	teamId	Relationship mapping	patientUid, caregiverUids[], doctorUids[], emergencyContactRefs[]
devices	deviceId	Device registry	patientUid, appVersion, modelPackVersion, lastSyncAt, lastActivityAt
gameCatalog	gameId	Versioned game definitions	gameVersion, domains[], itemRefs[], enabled
gameItems	itemId	IRT/adaptive item bank	gameId, domain, difficulty, discrimination, guessing, contentRefs
gameSessions	sessionId	Completed session summary	patientUid, gameId, score, domainScores, startedAt, completedAt, audioPath, schema_version
telemetry	eventId	Compact operational telemetry	patientUid, eventType, occurredAt, payload, schema_version
medications	medicationId	Medication definition	patientUid, label, instructions, schedule
medicationEvents	eventId	Medication action event	patientUid, medicationId, scheduledAt, actionAt, action
alerts	alertId	Welfare/operational alerts	patientUid, type, severity, createdAt, status, acknowledgedAt
auditLogs	logId	Clinician/data access audit	actorUid, role, targetPatientUid, action, timestamp
languagePacks	packId	Model pack metadata	language, version, storagePath, checksum, minAppVersion

This is a concrete implementation schema proposed for this documentation. The uploaded conversation establishes the data categories and services but does not prescribe these exact collection names, so the names above are a build reference rather than a claim about the prior prototype.

11. Firebase Security and Access Control

The architecture uses strict Firebase Security Rules with UID mapping, extended by Firebase Custom Claims for clinician role authorization. The key correction was that a naive `request.auth.uid == patientUid` policy would lock doctors out of patient data. The source resolves this by placing a role: doctor claim in verified clinician tokens and allowing clinician reads under that role.

11.1 Identity Lifecycle

36. Create the Firebase Auth account through an authorized provisioning path.
37. Associate the Auth UID with the patient, caregiver or doctor profile in Firestore.
38. For doctors, use an admin-only Cloud Function or controlled admin process to set the Custom Claim role: doctor after hospital verification.
39. Force token refresh in the client after role assignment before relying on claim-gated reads.
40. Remove or suspend the claim when clinician access is revoked.

11.2 Storage Rule Model

- Patient upload paths are owned by the recording patient/device identity.
- Patient can read only the patient-owned path.
- Clinicians with the doctor claim may read clinically authorized patient paths according to the chosen policy.
- Model pack paths are read-only to registered app clients; writes are restricted to deployment/admin principals.
- No client stores an unrestricted raw `gs://` path as a public URL.

11.3 Firestore Rule Model

- Patients can write their own activity/session telemetry only where the device is linked to the patient.
- Caregivers can read/write configuration only for patients to whom they are assigned.
- Doctors can read assigned patients through role claim plus care-team membership; if the prototype uses the simpler universal doctor-read policy, the audit scope should still be logged.
- No client may change its own role, patient linkage, clinician status or emergency escalation recipients.
- Cloud Functions use privileged server-side credentials for administrative mutations and schema adaptation.

11.4 Security Controls

- TLS for transit; encryption at rest for managed Firebase data.
- Custom Claims for coarse-grained role, Firestore relationships for fine-grained assignment.
- Explicit auditLogs collection for clinician access to patient records and audio.
- No signed/download tokens are hard-coded into the web app or stored as permanent public links.

12. Schema Evolution and Server Functions

The source rejected implicit NULL-based evolution and requires an explicit `schema_version` field on all payloads. Cloud Functions and the dashboard act as adapters so an older Android client can safely write data into a newer backend structure.

12.1 Versioning Rules

- Every Room payload and Firestore document produced by the mobile client carries `schema_version`.
- Every major app release increments `client_version` separately from `schema_version`.
- Backend adapters map older payloads to canonical server objects.
- New fields must have explicit default semantics; never assume a missing field is equivalent to a clinically meaningful value.
- Rejected schemas are quarantined into a diagnostics path rather than dropped.

12.2 Required Cloud Functions

Function	Trigger	Purpose
setDoctorClaim	Admin action	Sets/removes role: doctor on verified clinician Auth account
normalizeSession	Firestore write / callable path	Validates <code>schema_version</code> and converts legacy payload to canonical shape
welfareHeartbeat	Scheduled function	Checks last activity/sync against configured inactivity policy
issueAlert	Internal event	Creates alert, de-duplicates repeated alerts and routes notifications
cleanupOrphans	Scheduled maintenance	Finds Storage blobs with no corresponding completed Firestore session
modelPackVerifier	Storage/admin flow	Verifies model metadata/checksum before making a pack deployable

12.3 Welfare Alert Logic

41. Each patient has a configured intended-activity schedule.
42. Each activity completion writes `lastActivityAt`; each successful cloud synchronization updates `lastSyncAt`.
43. If an intended event is missed, the mobile client records the missed event and schedules the lightweight inactivity signal path.
44. When a server-side heartbeat evaluates the patient, it compares the last known timestamps to the policy window.
45. If the patient remains beyond the escalation threshold, create a single active welfare alert and notify configured caregiver/doctor/emergency routes.
46. Do not create an alert storm: active unresolved alerts must be de-duplicated until acknowledged or cleared.

13. React + Vite Doctor Web Dashboard

The dashboard is intentionally simple in implementation: React + Vite, Firebase JS SDK, Firestore queries, Firebase Auth and browser-side audio decoding. The value is in clear clinical presentation, not backend complexity.

13.1 Dashboard Information Architecture

Screen	Primary content	Actions
Login	Doctor Auth, role state, hospital/clinic identity	Sign in, sign out
Overview	Patient list, active alerts, adherence snapshot, last sync	Open patient, filter alert state
Patient Summary	Profile, care team, medication status, cognitive trend	Open detail tabs
Cognitive Trends	Domain scores, recent sessions, response latency, item difficulty trend	Date range, domain filter
Session Detail	Item-level responses, score breakdown, adaptive context, audio link	Play audio, inspect session
Medication	Scheduled vs taken vs missed, adherence trend	Filter, acknowledge note
Alerts	Open welfare alerts, timestamps, acknowledgment status	Acknowledge/escalate
Device Health	Last sync, queue health, storage status, model pack version	Flag device for clinic review
Audit	Doctor access events and changes	Filter/export for authorized administrators

13.2 Data Cards

- Current welfare state: normal / watch / alert.
- Last known patient activity and last cloud synchronization.
- 7-day and 30-day medication adherence.
- Game completion rate and session count.
- Domain-level cognitive trajectory, clearly labeled as app-derived monitoring rather than diagnosis.
- Response latency distribution to distinguish correctness changes from speed changes.
- Audio evidence count and playback availability.
- Device storage and language-pack health.

13.3 Clinical Trend Rules

The dashboard should show trends rather than a single score. Use a consistent baseline, session completeness indicator, and domain-specific history. Any graph must visibly distinguish missing/offline intervals from zero scores. Older schema data should be adapted before rendering so the UI never maps directly over unknown fields.

14. Dashboard Audio Playback

The final mobile format is raw AMR-WB because it is bandwidth-efficient and speech-friendly. Modern browser audio elements cannot be assumed to decode raw .awb files, so the dashboard must decode in the browser.

47. Authenticate the doctor with Firebase Auth.
 48. Call the Firebase JS SDK `getDownloadURL` on the protected Storage reference.
 49. Fetch the binary audio only after authorization succeeds.
 50. Pass the binary to a browser-side WASM decoder for AMR-WB.
 51. Convert the decoded data into a Web Audio API buffer or browser-playable representation.
 52. Provide play/pause, duration, loading and error states without exposing the raw Storage path to the user.
- The source explicitly adopts `getDownloadURL`, Google Cloud Storage CORS configuration and WASM AMR-WB decoding for the React/Vite dashboard.

14.1 CORS Deployment Contract

- Whitelists must include the production dashboard origin and approved development origin(s).
- No wildcard origin should be used for protected clinical data unless explicitly required and risk-reviewed.
- After changing the dashboard domain, update the bucket CORS configuration and regression-test audio playback.
- Access-control errors must be surfaced as authorization failures, not generic network failures.

15. End-to-End Data Flows

15.1 Patient Game Flow

53. AlarmManager launches the game reminder at the scheduled time.
54. Patient enters the game; storage gate verifies ≥ 50 MB free.
55. Adaptive engine selects a game and items locally.
56. Patient interacts primarily through MCQ/tap input with TTS option readout as needed.
57. One designated question may open the speech capture flow.
58. AudioRecord $\rightarrow$ 44.1 kHz PCM $\rightarrow$ Kotlin resampler $\rightarrow$ 16 kHz mono $\rightarrow$ Bhashini STT + MediaCodec AMR-WB.
59. Game session is finalized and written to Room as one or more telemetry packages.
60. Unique WorkManager sync is scheduled with CONNECTED constraint.
61. When connected, token-aware worker refreshes Auth if necessary, uploads audio if pending, then writes Firestore metadata.
62. Cloud heartbeat sees new lastActivityAt/lastSyncAt and the dashboard reflects the session.

15.2 Medication Flow

63. AlarmManager triggers the medication reminder.
64. Patient/caregiver confirms Taken, Snoozed or Unable-to-take.
65. Event is written locally immediately.
66. Sync is queued for Firestore when connectivity is available.
67. Caregiver/doctor dashboard updates after synchronization.
68. Welfare logic uses missed expected activity to contribute to its broader patient status policy.

15.3 Offline-to-Online Flow

69. Device loses connectivity; core patient operations continue.
70. Room queue accumulates records in sequence order.
71. No background network loop fires because WorkManager is constrained to CONNECTED.
72. Device reconnects to 2G/3G/Wi-Fi.
73. WorkManager wakes the sync worker; Auth token refresh is checked first.
74. FIFO queue drains with upload checkpoints and exponential backoff.
75. Dashboard receives normalized records; server heartbeat evaluates current welfare state.

16. Failure Handling and Resilience Matrix

Failure	Expected behavior	Persistent artifact	Recovery
No network for days	Patient app continues; queue stays local	Room telemetry	Automatic sync on reconnect
Auth token stale	Worker awaits getIdToken(false)	No record loss	Retry only if refresh fails
Audio upload succeeds, Firestore write fails	Persist AudioUploaded state	Storage path in Room	Retry metadata only
Worker duplicated after missed activities	Unique Work replaces duplicate job	Single pending job	No thundering herd
Disk < 50 MB	Block game launch; caregiver warning	Diagnostic state	Free storage / retry
TFLite operator unsupported	Reject pack/app start for affected feature	Pack validation log	Deploy corrected model/runtime
44.1 kHz hardware, model expects 16 kHz	Resample in software	No corrupted tensor	Feed only resampled 16 kHz buffer
AMR-WB browser unsupported	Decode in WASM	Audio remains in Storage	Play decoded Web Audio buffer
Legacy payload schema	Cloud adapter transforms data	Original schema_version retained	Render canonical structure
Partial model-pack download	Mark invalid and do not activate	Checksum failure record	Redownload at clinic
Repeated welfare condition	Create one active alert, suppress duplicates	Alert status/ack state	Caregiver/doctor acknowledgement
Unexpected app shutdown during game	Resume or finalize from local state	Room game state	Continue without duplicate submission

17. Observability, Audit and Operations

17.1 Device Health Metrics

- Last local activity timestamp and last cloud sync timestamp.
- Pending queue count and oldest pending age.
- Upload success/failure count and retry count.
- Free storage bytes and storage-block events.
- Battery band and app uptime health.
- Installed language pack version and checksum validation status.
- Speech inference success/failure and average inference duration.
- Game completion/abandonment counts.

17.2 Server Metrics

- Cloud Function success/error rate.
- Welfare heartbeat execution and alert creation rate.
- Firestore write rejection rate by schema_version and role.
- Storage upload orphan count and cleanup backlog.
- Dashboard authorization failures and audio playback failures.
- Active unresolved welfare alerts by clinic.

17.3 Audit Requirements

- Doctor reads of patient records and audio should be auditable.
- Role assignment and revocation should be audited.
- Language-pack deployment and model updates should be auditable.
- Backend schema-adapter conversions should record source schema_version and target version.
- Alert acknowledgement and escalation should be auditable.

18. Clinical Safety, Privacy and Data Governance

This platform handles sensitive health-adjacent information. Industry-grade implementation requires least privilege, durable audit trails, data minimization, controlled retention, and an explicit separation between product-derived monitoring and medical diagnosis.

18.1 Minimum Controls

- No public Storage objects containing patient audio.
- No unrestricted Firestore read access from the dashboard.
- No client-side role elevation.
- No permanent public download URLs for protected audio.
- No clinical interpretation of an isolated game score.
- Clear retention policy for audio and telemetry; keep only what is operationally and clinically justified.
- Clock, network and offline state must be interpreted with uncertainty where device conditions make timestamps unreliable.

18.2 Welfare Alert Language

A prolonged lack of connectivity or activity is a welfare signal, not proof of an emergency. The system intentionally tolerates false positives because a human welfare check is safer than silently assuming the patient is safe. Dashboard copy should say that the alert requests verification rather than claiming a medical emergency has occurred.

19. Complete Decision and Correction History

This section preserves the evolution of the architecture so the implementation team understands not only what is final, but why earlier alternatives were removed.

Decision area	Earlier approach	Final decision	Reason/source
Initial deployment contract	Clinic/hospital setup after doctor + caregiver contact	Locked	Source pp. 1-4
Storage persistence	RoomDB stored as app-data, not volatile cache	Locked	Source pp. 1, 27-30
Audio capture limit	5-second STT ceiling	Removed	Clinical timing and truncation risk identified
Game interaction model	STT for every question	Removed	Replaced by MCQ-heavy games plus one speech question
Speech question window	5 seconds	Changed to 15-30 seconds	Supports slower response without multiplying audio uploads
Heartbeat concept	No client inactivity ping	Added 15-30 minute missed-activity signal path	Server-side welfare watchdog introduced
WorkManager pings	Independent enqueue per missed session	Changed to unique work / REPLACE	Prevents thundering herd
Auth sync	Write immediately after reconnect	Changed to token-aware CoroutineWorker	Await getIdToken(false) before protected writes
Audio format	AAC at 128 kbps with ~15 KB target	Rejected	File-size math incompatible with 15-30 seconds
Audio format	Custom C/JNI LAME MP3 then Media3 to AAC	Rejected	Double lossy encoding and extra battery cost
Audio capture	Direct 44.1 kHz into Bhashini	Rejected	Model expects 16 kHz mono
Resampling	Rely on hardware / OS	Changed	Pure Kotlin linear interpolation 44.1 kHz -> 16 kHz
Final mobile audio	AAC	Changed	AMR-WB 15.85 kbps
Container packaging	MediaMuxer for raw AMR-WB	Rejected	Raw AWB container must be framed directly
Web playback	HTML5 audio direct	Rejected	Browser AMR-WB support unavailable
Web playback	Firebase getDownloadURL + CORS + WASM decoder	Locked	Final path
Doctor access	UID-only Storage rule	Extended	Custom Claim role: doctor
Model runtime	Standard TFLite only	Extended	tensorflow-lite-select-tf-ops
Model distribution	Download precompiled Bhashini binaries from external CDN	Rejected	Self-host compiled mobile binaries in Firebase Storage
Long-term disk	Assume storage never fills	Changed	50 MB pre-flight storage interceptor
Schema evolution	Implicit NULLs in JSON	Rejected	Explicit schema_version + backend adapters
Background battery	Retry worker without connectivity constraint	Changed	WorkManager NetworkType.CONNECTED
Reminder execution	WorkManager for everything	Changed	AlarmManager for exact reminders, WorkManager for network sync

20. Build Specification for the Implementation Team

This section converts the architecture into a hand-build order. It does not provide source code; it defines interfaces, responsibilities, acceptance criteria and dependency order so a Kotlin/React/Firebase team can implement the system without discovering the architecture during coding.

20.1 Phase A - Android Foundation

76. Create the Kotlin/Compose application and establish navigation, theming, accessibility and application lifecycle handling.
77. Add Room with versioned migrations and durable application-data storage.
78. Implement Patient/Device configuration and secure local session state.
79. Implement AlarmManager-based reminders.
80. Implement storage safety gate and caregiver-facing storage management flow.
81. Implement basic game engine abstractions and session persistence before adding any adaptive intelligence.

20.2 Phase B - Game Engine

82. Create a common GameDefinition model and version it.
83. Create item content models with accessibility metadata and TTS text.
84. Implement MCQ selection/highlighting/green submit interaction.
85. Implement timing capture and retry/resume semantics.
86. Implement at least 6 representative games spanning memory, attention, executive, visuospatial, language and orientation before expanding the catalog.
87. Add session-level Room persistence and deterministic local IDs.

20.3 Phase C - Adaptive Intelligence

88. Implement item-bank lookup and IRT parameter storage.
89. Implement Bayesian latent estimate update.
90. Implement candidate filtering and policy constraints.
91. Add bounded contextual-bandit selection.
92. Persist adaptive state locally and test deterministic recovery after interruption.
93. Only then activate adaptive selection in the daily game launcher.

21. Build Specification - Speech and Sync

21.1 Phase D - Speech Runtime

94. Integrate TFLite with select TensorFlow ops.
95. Implement language-pack loading and validation.
96. Implement TTS service facade first, because it is needed across all patient screens.
97. Implement AudioRecord capture with native sample-rate detection and a fixed mono channel policy.
98. Implement pure Kotlin resampling to 16 kHz.
99. Integrate Bhashini STT and validate tensor shapes with device-level fixtures.
100. Implement MediaCodec AMR-WB encoding from the resampled 16 kHz stream.
101. Implement direct raw AWB framing and file integrity validation.

21.2 Phase E - Sync Engine

102. Implement TelemetryPackage Room entity with upload_state and schema_version.
103. Implement FIFO DAO ordering.
104. Implement token-aware CoroutineWorker.
105. Add NetworkType.CONNECTED constraint and exponential backoff.
106. Implement unique work for queue draining and inactivity signal jobs.
107. Implement Storage upload checkpoint followed by Firestore metadata write.
108. Implement terminal completion and replay tests under forced network drops.

21.3 Acceptance Tests

Test	Pass condition
Offline game	Game completes and remains fully represented in Room with no network
Power loss	Restarting app preserves the session/queue state
Audio drop	Storage success + Firestore failure does not re-upload audio
Auth expiry	Expired token refresh happens before protected writes
Zero network	Sync worker does not run until connectivity exists
Storage low	Game is blocked below 50 MB free space
Model mismatch	Invalid language pack never becomes active
Schema v1 to v2	Legacy payload renders safely in dashboard
Doctor access	Assigned doctor can read authorized patient data; unauthorized role cannot
Audio playback	Doctor can hear protected AMR-WB recording in supported browser build

22. Build Specification - Web Dashboard

22.1 React/Vite Structure

Area	Responsibility
auth	Firebase Auth state and doctor claim verification
queries	Firestore data access hooks and query builders
adapters	Schema-version normalization before rendering
state	Selected patient, filters, date range, alert state
components	Cards, tables, trend charts, session viewer, audio player
audio	getDownloadURL, fetch, WASM decode, Web Audio playback
audit	Access logging and acknowledgement actions
error boundary	Human-readable handling for authorization, missing fields, offline dashboard state

22.2 Dashboard Data Loading Rules

- Never render raw Firestore documents directly; normalize them through a version adapter.
- Use small, purpose-built queries rather than downloading full patient histories.
- Paginate session histories and audit logs.
- Filter on server-supported indexes where possible.
- Display an explicit "data incomplete / awaiting sync" indicator when the patient has been offline.

22.3 Recommended Indexes

- gameSessions by patientUid + completedAt descending.
- medicationEvents by patientUid + actionAt descending.
- alerts by patientUid + status + createdAt descending.
- auditLogs by targetPatientUid + timestamp descending.
- telemetry by patientUid + occurredAt descending where detailed drill-down is required.

23. Build Specification - Firebase Project Provisioning

- 109. Create separate Firebase projects/environments for development, staging and production.
- 110. Enable Firebase Authentication with the selected login methods.
- 111. Create Firestore database and establish locked collection/document naming conventions.
- 112. Create Storage buckets/paths for patient audio, language packs and quarantine/orphan paths.
- 113. Deploy Security Rules and test them using emulator or controlled integration tests before any clinical data is present.
- 114. Create Cloud Functions for Custom Claims, schema normalization, welfare heartbeat, alert routing and maintenance.
- 115. Configure Cloud Storage CORS for approved dashboard origins.
- 116. Create and document FCM/SMS routing identities and alert preferences.
- 117. Populate game catalog, item bank, language pack metadata and version registry.
- 118. Enable billing controls, quotas, logs, alerts and scheduled maintenance policies before pilot deployment.

23.1 Firestore Write Contract

Every mobile write should be self-describing enough for backend adaptation: `schema_version`, `client_version`, `patientUid`, `deviceId`, `event/session id`, `occurredAt`, and a stable event type. Backend functions should treat unknown schema versions as a controlled validation error. They must never infer clinical meaning from malformed data.

23.2 Storage Path Contract

Use deterministic names that prevent collisions and support cleanup, for example `patient-audio/{patientUid}/{yyyy}/{mm}/{sessionId}.awb` and `language-packs/{language}/{packVersion}/...tflite`. The exact path names are implementation choices, but the invariant is that each session has one canonical audio object and the Firestore record stores the logical storage path, not a permanent public URL.

24. Recommended Corporate-Level Enhancements

These items are not presented as prior conversation decisions. They are implementation-grade extensions recommended by this specification to make the locked architecture easier to operate at scale.

- Formal release train with schema compatibility matrix across mobile, dashboard and Cloud Functions.
- Configuration-driven feature flags so games, language packs and thresholds can be enabled per clinic without app redeployment.
- Automated device acceptance test suite for target Android hardware including microphone, storage, alarms, TFLite and 2G/3G network simulation.
- Golden audio fixtures for each supported language to detect STT degradation after model or codec changes.
- Clinical content review workflow for every new game item, including readability, cultural familiarity and bias review.
- Alert de-duplication and escalation ladders with time-based acknowledgement states.
- Offline export/repair path for devices that cannot reconnect for extended periods.
- Centralized incident runbook covering queue corruption, bad model pack, security rule regressions and failed Cloud Functions.

24.1 Corporate Readiness Gates

Gate	Required evidence
Security	Rules tests, role tests, audit logs, penetration review
Clinical UX	Caregiver/clinician usability validation, accessibility checks
Offline	Multi-day disconnected soak test with forced power loss
Speech	Language-level accuracy and intelligibility regression suite
Sync	Network-drop matrix including every upload state transition
Scalability	Firestore/Storage cost and query-load test
Release	Backward compatibility test for at least previous supported schema

25. Source Traceability and Interpretation Notes

To avoid silently rewriting history, this document uses a provenance rule. Statements labeled "locked" or "source-derived" come from the supplied conversation. Concrete collection names, extra dashboard cards, the expanded game catalog, and operational refinements that were not explicitly present in the source are labeled as proposed implementation details.

25.1 Key Source Anchors

Topic	Source anchor
Clinic provisioning, app-data Room persistence	Supplied PDF pp. 1-4
WorkManager inactivity signal + Auth token race solution	Supplied PDF pp. 2-4
MCQ-heavy games + 15-30 second speech response	Supplied PDF pp. 6-7
Idempotent upload_state state machine	Supplied PDF pp. 11-12
TFLite select ops + self-hosted language packs	Supplied PDF pp. 17-18
Single-pass audio capture and abandonment of double encoding	Supplied PDF pp. 24-26
50 MB storage gate + schema_version	Supplied PDF pp. 28-30
AMR-WB, 16 kHz resampling, WASM playback, Custom Claims	Supplied PDF pp. 35-47
Final 30-dimension locked inventory	Supplied PDF pp. 48-56

25.2 Final Locked Architecture Checklist

- Kotlin + Jetpack Compose
- High-contrast accessibility UI
- Patient/Caregiver/Doctor/Emergency separation
- React + Vite dashboard
- Firebase Storage for audio/model packs
- Firestore for structured telemetry
- Option C security + Custom Claims
- No on-device generative LLM
- Local Kotlin RL-IRT hybrid
- Bhashini IndicConformer + FastPitch/HiFi-GAN
- 150-300 MB clinic language packs
- Room in app-data
- FIFO queue
- Unique Work with REPLACE for inactivity queue control
- Token-aware CoroutineWorker with getIdToken(false)
- Cloud Function welfare heartbeat
- 15-30 second speech window
- MCQ-dominant games
- TTS + green Submit accessibility interaction
- upload_state idempotency
- tensorflow-lite-select-tf-ops
- Local PyTorch -> TFLite quantization pipeline
- Self-hosted model binaries
- AudioRecord + Kotlin 44.1 kHz -> 16 kHz resampling
- AMR-WB 15.85 kbps
- Raw AWB header framing
- 50 MB storage safety gate

- `schema_version` adapters
- GCS CORS + `getDownloadURL` + WASM
- `NetworkType.CONNECTED` for sync, `AlarmManager` for exact reminders

26. One-Page Implementation Blueprint

The implementation team should think of the system as five planes. The patient experience plane is Kotlin/Compose and must remain functional without internet. The intelligence plane is local RL-IRT plus local speech. The persistence plane is Room and must never lose completed session state merely because the device is offline. The transport plane is WorkManager plus Auth-aware, idempotent state-machine synchronization. The cloud plane is Firebase Auth, Firestore, Storage and Cloud Functions with strict role and relationship enforcement. The React/Vite dashboard is a read-heavy clinical presentation layer over the normalized Firestore model, with browser-side audio decoding for AMR-WB.

26.1 Non-Negotiable Invariants

- 119. No clinical session is considered complete until it is durably represented in Room.
- 120. No protected cloud write is attempted until the Auth token is usable.
- 121. No audio is re-uploaded after a successful Storage checkpoint.
- 122. No background sync job is allowed to wake a zero-connectivity device unnecessarily.
- 123. No game starts when local storage is below 50 MB.
- 124. No speech model receives a 44.1 kHz buffer directly; only the resampled 16 kHz mono stream goes to Bhashini.
- 125. No browser assumes native AMR-WB support; the dashboard decodes it explicitly.
- 126. No schema field is assumed to exist merely because a newer client expects it.
- 127. No welfare alert is treated as a diagnosis; it is a prompt for human verification.
- 128. No role is self-assigned by the client application.

26.2 Build Outcome

A team that follows this specification should be able to implement the system without needing to rediscover the architectural decisions that were made during the earlier audit rounds. The major corrections are preserved as hard constraints, while the game catalog and reference schema are explicitly identified as product-level implementation content added to make the blueprint executable at team scale.

Appendix A - Glossary

Term	Meaning
IRT	Item Response Theory, used to select/score items against a latent ability estimate
RL / contextual bandit	Adaptive policy mechanism that chooses among eligible interactions using context and reward signals
TTS	Text-to-speech
STT	Speech-to-text
PCM	Raw pulse-code modulation audio samples
TFLite	TensorFlow Lite mobile inference format
WASM	WebAssembly
FIFO	First-in, first-out
RBAC	Role-based access control
Custom Claim	Firebase Auth token attribute used for role authorization
schema_version	Explicit payload version for backward-compatible processing
AMR-WB	Speech-oriented audio codec selected for the final low-bandwidth architecture

Appendix B - Delivery Checklist

- Android app builds, installs and starts on target tablet.
- Clinic provisioning completes successfully with selected language pack.
- All core reminders work offline.
- At least six games complete offline and persist to Room.
- One speech question per game records correctly within 15-30 seconds.
- AMR-WB file is human-audible and playable in dashboard through WASM.
- Queue survives app restart and device restart.
- Audio and metadata retry independently without duplicates.
- Firebase rules block unauthorized role access.
- Doctor dashboard loads normalized legacy and current schemas.
- Cloud welfare heartbeat creates de-duplicated alerts.
- Soak test passes multi-day offline scenario without sync-induced battery drain.
- Production logging and audit trails are enabled before pilot deployment.

End of specification.